

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Case Study

COURSE NAME: COMPUTER VISION

COURSE CODE:ITA0515

Slot B

COURSE OUTCOME COVERED

CO3: Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4)

Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks 50

Duration :60 Minutes

Annexure A

Case Study

Code of Conduct:

I _____ RUTHIKA REDDY (192421138)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly	
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts	
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning	
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided	
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand	

1. Preprocessing and Feature Detection

Case Analysis

Images captured under different lighting conditions may have variations in **brightness, contrast, shadows, and illumination**. These variations can change pixel intensities and cause the feature detector to identify different features in the same object.

Application of Concepts

Image preprocessing prepares the image before feature detection. Important preprocessing techniques include:

- **Grayscale conversion:** Reduces the image to intensity information.
- **Noise filtering:** Gaussian or median filtering removes unwanted noise.
- **Histogram equalization:** Improves overall image contrast.
- **CLAHE:** Improves local contrast, especially under uneven lighting.
- **Normalization:** Makes intensity values more consistent.

Analysis

Feature detectors such as **Harris Corner Detector, SIFT, or ORB** depend on intensity variations. If illumination changes significantly, the same feature may have different intensity values in different images. This can lead to:

- Missing features.
- False features.
- Inconsistent feature locations.
- Poor feature matching.

For example, a corner visible in a well-lit image may become difficult to detect when it is affected by a dark shadow.

Solution Design

A suitable pipeline is:

Input Image → Noise Removal → Grayscale → Contrast Enhancement → Normalization → Feature Detection

For uneven illumination, **CLAHE** can be used instead of simple histogram equalization. The preprocessing method should be selected according to the type of lighting and noise present.

Conclusion

Preprocessing improves feature detection by reducing the effects of **noise and illumination variation**. It produces more stable and reliable features, thereby improving applications such as **image matching, object recognition, and tracking**.

2. Edge vs Corner Detection

Case Analysis

The system uses edge detection but fails to identify important points for image matching. The main problem is that **edges represent boundaries**, but they do not always provide unique locations for matching.

Application of Concepts

Edge detection identifies regions with strong intensity changes. Examples include:

- Canny
- Sobel
- Prewitt

Corner detection identifies points where intensity changes significantly in **multiple directions**. Examples include:

- Harris Corner Detector
- Shi-Tomasi

Analysis

A straight edge contains many similar pixels along its length. Therefore, it is difficult to determine the exact location of a matching point.

A corner, however, is more distinctive because the intensity changes in two directions.

For example:

Straight edge:



Corner:



A corner provides a more unique feature than a straight edge.

Solution Design

The system should use a combination such as:

Image → Noise Reduction → Edge/Corner Analysis → Harris or Shi-Tomasi → Feature Matching

Corners should be selected as key points because they are more distinctive and suitable for matching.

For complex textured images, **SIFT or ORB** can also be used to obtain robust keypoints and descriptors.

Conclusion

Edge detection alone is insufficient for feature matching because edges may not provide unique keypoints. **Corner detection identifies distinctive locations**, making it more effective for image matching, tracking, and object recognition.

3. Hough Transform in Noisy Images

Case Analysis

The Hough Transform detects lines by accumulating votes for possible lines. In a noisy image, unwanted pixels may be interpreted as edges and generate incorrect votes.

Application of Concepts

The Hough Transform generally works as:

Image → Edge Detection → Hough Transform → Line Detection

For a line, the Hough Transform represents it using parameters such as:

$$\rho = x \cos \theta + y \sin \theta$$

where:

- ρ = distance from the origin.
- θ = angle of the line.

Analysis

Noise introduces additional edge pixels. These pixels can produce:

- False peaks in Hough space.
- Incorrect lines.
- Multiple unwanted lines.
- Reduced detection accuracy.

If the Hough threshold is too low, even weak random patterns may be detected as lines.

Solution Design

The system can be improved using:

1. **Gaussian/Median filtering** – reduces noise.
2. **Canny edge detection** – produces cleaner edges.
3. **Proper Canny thresholds** – removes weak unwanted edges.
4. **Hough threshold adjustment** – removes insignificant lines.
5. **Suitable θ and ρ resolution** – improves accuracy.
6. **Probabilistic Hough Transform** – can efficiently detect significant line segments.

A robust pipeline is:

Noisy Image → Gaussian Filter → Canny Edge Detector → Hough Transform → Thresholding → Final Lines

Conclusion

Noise reduces Hough Transform accuracy by generating false edge points and votes. Combining **noise removal, good edge detection, and appropriate Hough parameters** significantly improves line detection.

4. Feature Matching under Transformations

Case Analysis

Two images contain the same object but are captured at different **scales and orientations**. Traditional feature matching may fail because the appearance and positions of features change.

Application of Concepts

Traditional feature descriptors may not be sufficiently invariant to transformations.

Scale-invariant feature methods, such as **SIFT**, are designed to identify features across different scales.

SIFT provides:

- Scale-space feature detection.
- Orientation assignment.
- Local feature descriptors.
- Robust feature matching.

Analysis

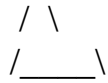
Consider the same object:

Image 1:

Object
^
/ \
/____\
|

Image 2:

^



Larger / rotated

Although the object is the same, its feature coordinates and appearance change.

A scale-invariant method searches for important features at **multiple scales** and assigns an orientation to each feature. Therefore, the descriptor remains more consistent when the object is resized or rotated.

Solution Design

A suitable approach is:

Image → Scale-Space Detection → Keypoint Detection → Orientation Assignment → Descriptor Generation → Feature Matching

Methods such as **SIFT** are particularly effective for scale and rotation changes. **ORB** is another efficient alternative for many real-time applications.

Conclusion

Scale-invariant feature matching performs better because it is designed to maintain feature consistency despite **changes in size and orientation**. This makes it useful in **object recognition, image stitching, panorama creation, and tracking**.

5. PCA for Feature Optimization

Case Analysis

The system extracts a large number of features, causing:

- High computational cost.
- Increased memory usage.
- Redundant information.
- Longer processing time.
- Potential difficulty in training machine-learning models.

Application of Concepts

Principal Component Analysis (PCA) is a dimensionality-reduction technique that transforms correlated features into a smaller set of **principal components**.

PCA identifies directions that contain the maximum variance in the data.

Analysis

Suppose an image system extracts **100 features**. Some features may contain almost the same information.

PCA can transform these 100 features into, for example, **20 principal components** while retaining most of the important information.

The main steps are:

Feature Extraction → Standardization → Covariance Matrix → Eigenvalues/Eigenvectors → Select Principal Components → Reduced Features

Components with higher eigenvalues contain more variance and are generally selected first.

Solution Design

The system should apply PCA after feature extraction:

Original Features → PCA → Reduced Features → Classification/Matching

The number of components can be selected based on the desired **explained variance**, such as 90%, 95%, or 99%.

Benefits

PCA:

- Reduces dimensionality.
- Removes correlated/redundant information.
- Decreases computation.
- Reduces memory requirements.
- Speeds up machine-learning algorithms.
- Can reduce overfitting in some applications.

Conclusion

PCA provides an efficient representation of high-dimensional features by retaining the most important information while removing redundancy. Therefore, it improves **computational efficiency and system performance**.